

Nile University (NU) IoT Research Internship Report

Day 8

Overview

Day 8 focused on establishing wired communication between the CC1352P7 LaunchPad and the Raspberry Pi using UART over USB. An ultrasonic sensor was connected to the CC1352P7, distance measurements were read on the microcontroller, and the data was transmitted via UART to the Raspberry Pi for display.

Task 1: Ultrasonic Sensor Connection and Reading on CC1352P7

Connected the HC-SR04 ultrasonic sensor to the CC1352P7 LaunchPad, wiring VCC to 5V, GND to GND, TRIG to GPIO DIO13, and ECHO to GPIO DIO14. Configured the corresponding GPIO pins and implemented distance measurement code on the CC1352P7.

This task moved the same HC-SR04 sensor work done earlier on the Raspberry Pi (Day 2) onto the CC1352P7 microcontroller instead, confirming that the same sensing principle could be implemented on the embedded platform introduced on Days 5 through 7.

Task 2: UART Configuration and Data Formatting

Configured UART on the CC1352P7 for serial communication, with the data formatted as a CSV-style string ("distance,temperature,humidity") for easy parsing on the receiving end.

Task 3: Wired Connection to Raspberry Pi

Connected the CC1352P7 to the Raspberry Pi via a USB cable, which functions as a UART link with the CC1352P7 appearing as a virtual COM port (/dev/ttyUSB0 or /dev/ttyACM0). A baud rate of 115200 was used and had to match on both devices.

Improvement over Task 2: Task 2 got the CC1352P7 ready to transmit data, but there was no receiving device yet. This task physically linked the two platforms used throughout the internship so far, the Raspberry Pi (Days 1-4) and the CC1352P7 (Days 5-7), for the first time.

Task 4: Reading and Displaying Data on Raspberry Pi

Wrote a Python script on the Raspberry Pi to read the incoming serial data and display the distance readings clearly on the terminal. Verified that data was transmitted and received correctly in real time.

Improvement over Task 3: Task 3 established the physical and serial connection, but without a program to read and interpret it, the data was inaccessible. This task closed the loop, completing an end-to-end path from sensor, to microcontroller, to UART transmission, to readable output on the Raspberry Pi.

```
receiving trigger for malicup (0.10.10.10) 1.1.1.1
(.IoT_venv) Intern@Intern:~/tasks $ minicom -D /dev/ttyACM0 -b 115200

25,60
Welcome to minicom 2.8
      I
OPTIONS: I18n
Port /dev/ttyACM0, 17:11:13

Press CTRL-A Z for help on special keys
```

Data Flow

1. CC1352P7 sends a trigger pulse to the ultrasonic sensor
2. Sensor returns an echo pulse, with pulse width corresponding to distance
3. CC1352P7 calculates distance in centimeters
4. CC1352P7 formats the data as "distance,temperature,humidity"
5. Data is sent via UART over USB
6. Raspberry Pi reads the serial data
7. Distance is displayed on the terminal

Commands Used

```
# Check serial port
ls -l /dev/ttyUSB*
ls -l /dev/ttyACM0

# Add user to dialout group for permissions
```

```
sudo usermod -a -G dialout pi
```

```
# Run Python receiver script  
python3 read_uart.py
```

Key Learnings

- UART communication: serial data transmission over USB between two independent devices
- Baud rate: must match on both ends of the connection (115200 used here)
- Virtual COM port: the CC1352P7 appears to the Raspberry Pi as /dev/ttyUSB0 or /dev/ttyACM0
- Ultrasonic distance calculation: $\text{distance_cm} = \text{pulseWidth} / 58$
- Data formatting: using CSV-style strings makes parsing simpler on the receiving side

Tools & Technologies

- IDE: Code Composer Studio (TI)
- Hardware: CC1352P7 LaunchPad, HC-SR04 sensor, Raspberry Pi
- Communication: UART over USB
- Languages: C (CC1352P7), Python (Raspberry Pi)

Outcome

- Connected an ultrasonic sensor to the CC1352P7 and read distance measurements on it
- Configured and established UART communication over USB between the CC1352P7 and Raspberry Pi
- Successfully transmitted sensor data from the microcontroller to the Raspberry Pi
- Displayed real-time distance readings clearly on the Raspberry Pi terminal
- Bridged the two hardware platforms used throughout the internship into a single working system